

Machine Learning Project

Final Report — Milestones 1 & 2

Online Games Popularity Prediction

Team #19

Preprocessing • Feature Selection • Regression Modeling • Classification

Team Members:

1. Raafat Mahmoud Abd El-Azim Marzouk	2023170211
2. Khattab Abd El-Moniem Khattab Mohamed	2023170204
3. Ahmed Eid Awad Ali	2023170048
4. Abd El-Rahman Mosa Abd El-Hakam Mosa	2023170347
5. Zainab Alaa Ali Ibrahim	2023170249
6. Youssef El-Sayed Mohamed Ahmed	2023170715

1. Introduction

The goal of this project is to predict how popular an online game will be on the Steam platform. Popularity is quantified across two milestones: in Milestone 1 by the **RecommendationCount** — a numeric score requiring regression — and in Milestone 2 by the **GamePopularity** label (Low, Medium, or High), a multi-class classification task. Understanding what drives game popularity can help developers and publishers make better decisions at the design and marketing stages.

2. Dataset Overview

The dataset was created by merging **train_data.csv** and **idlist.csv** on the QueryID field. The merged dataset contains approximately **13,000+ rows** and more than **70 columns** spanning numeric, boolean, date, and text-type features.

Key feature categories include:

- Popularity metrics: SteamSpyOwners, SteamSpyPlayersEstimate, and their variance columns.
- Game content: AchievementCount, AchievementHighlightedCount, ScreenshotCount, MovieCount, DLCCount.
- Pricing: PriceInitial, PriceFinal, IsFree, GenrelsFreeToPlay.
- Genre flags (15+): GenrelsIndie, GenrelsAction, GenrelsAdventure, GenrelsRPG, etc.
- Platform flags: PlatformWindows, PlatformLinux, PlatformMac.
- Category flags: CategoryMultiplayer, CategoryCoop, CategoryMMO, CategorySinglePlayer, etc.
- Temporal: ReleaseDate (engineered into ReleaseYear and ReleaseMonth).
- Text / URL columns (dropped): ShortDescrip, AboutText, HeaderImage, Website, etc.

For Milestone 2, the RecommendationCount target column was removed and replaced with the **GamePopularity** column (Low, Medium, High).

3. Preprocessing

All features were preprocessed before any feature selection or modeling. The same pipeline was applied to both the regression and classification datasets. The following steps were applied in order:

3.1 Dropping Irrelevant Features

Free-text fields (*ShortDescrip*, *AboutText*, *DetailedDescrip*, *Reviews*, *LegalNotice*, *SupportedLanguages*, *Website*, *HeaderImage*), URL columns, and identifier columns (*QueryID*, *ResponseID*, *GameName*, *ID*) were dropped. These carry no numeric signal; processing raw text would require NLP pipelines beyond the scope of this project. The **PriceCurrency** column also dropped because it was a constant value (USD) across all rows, contributing zero variance. A total of 25 columns were removed in this step.

3.2 Temporal Feature Engineering

The *ReleaseDate* column was parsed using `pandas.to_datetime()` with `errors="coerce"` to handle malformed entries. Two new numerical features were extracted: **ReleaseYear** and **ReleaseMonth**. *ReleaseYear* captures generational market trends (older vs. modern games), while *ReleaseMonth* captures seasonal release patterns (e.g., holiday launches). The original *ReleaseDate* column was then dropped.

3.3 Boolean Encoding

All 35 boolean columns (genre flags, platform flags, and category flags) were standardised to integer values of 0 and 1. The mapping handled multiple formats found in the raw data: Python True/False, string "TRUE"/"FALSE", and numeric 1/0. Missing values in boolean columns were filled with 0, representing the absence of the characteristic.

3.4 Median Imputation

Seventeen numeric columns (*SteamSpy estimates*, achievement counts, prices, screenshot counts, etc.) were imputed using the **median strategy** via sklearn's `SimpleImputer`. The median was preferred over the mean because these columns exhibit heavy right-skew — the median is robust to extreme outliers and avoids inflating imputed values.

3.5 IQR Outlier Capping

Extreme outliers in numeric features were capped using the **Interquartile Range (IQR) method**. For each numeric column, values below $Q1 - 1.5 \times IQR$ were clipped to the lower fence, and values above $Q3 + 1.5 \times IQR$ were clipped to the upper fence. This prevents extreme *SteamSpy estimate* outliers from distorting regression slope estimates while preserving the overall distribution shape.

3.6 Log1p Transformation

Five columns — *SteamSpyOwners*, *SteamSpyOwnersVariance*, *SteamSpyPlayersEstimate*, *SteamSpyPlayersVariance*, and the Milestone 1 target **RecommendationCount** — were log1p-transformed (i.e., $\log(1 + x)$). These columns span many orders of magnitude (from zero to millions), violating the linearity and homoscedasticity assumptions of linear regression. Log compression brings these into a manageable range. Using log1p instead of plain log ensures safe handling of zero values. The inverse transformation `np.expm1()` is applied at inference time to convert log-space predictions back to raw counts.

4. Feature Analysis

Feature selection was performed using sklearn's **SelectKBest** with the **f_regression** scoring function for Milestone 1 and **f_classif** for Milestone 2. Both functions measure the statistical relationship between each feature and the target using an F-test. The top 25 features were ranked and the top 20 were selected for model training in each phase.

4.1 Regression Feature Importance (Milestone 1)

The four SteamSpy columns (PlayersEstimate, Owners, PlayersVariance, OwnersVariance) dominate the rankings by a wide margin, with F-scores ranging from 3,948 to 6,651. ReleaseYear follows at F-Score 2,161, and AchievementCount at 1,693. Price columns, screenshot/movie counts, multiplayer flags, and system requirements round out the selected top 20 features.

Figure 1 — Top 25 Feature F-Scores (Regression) and Table 1 — Exact F-Score Values:

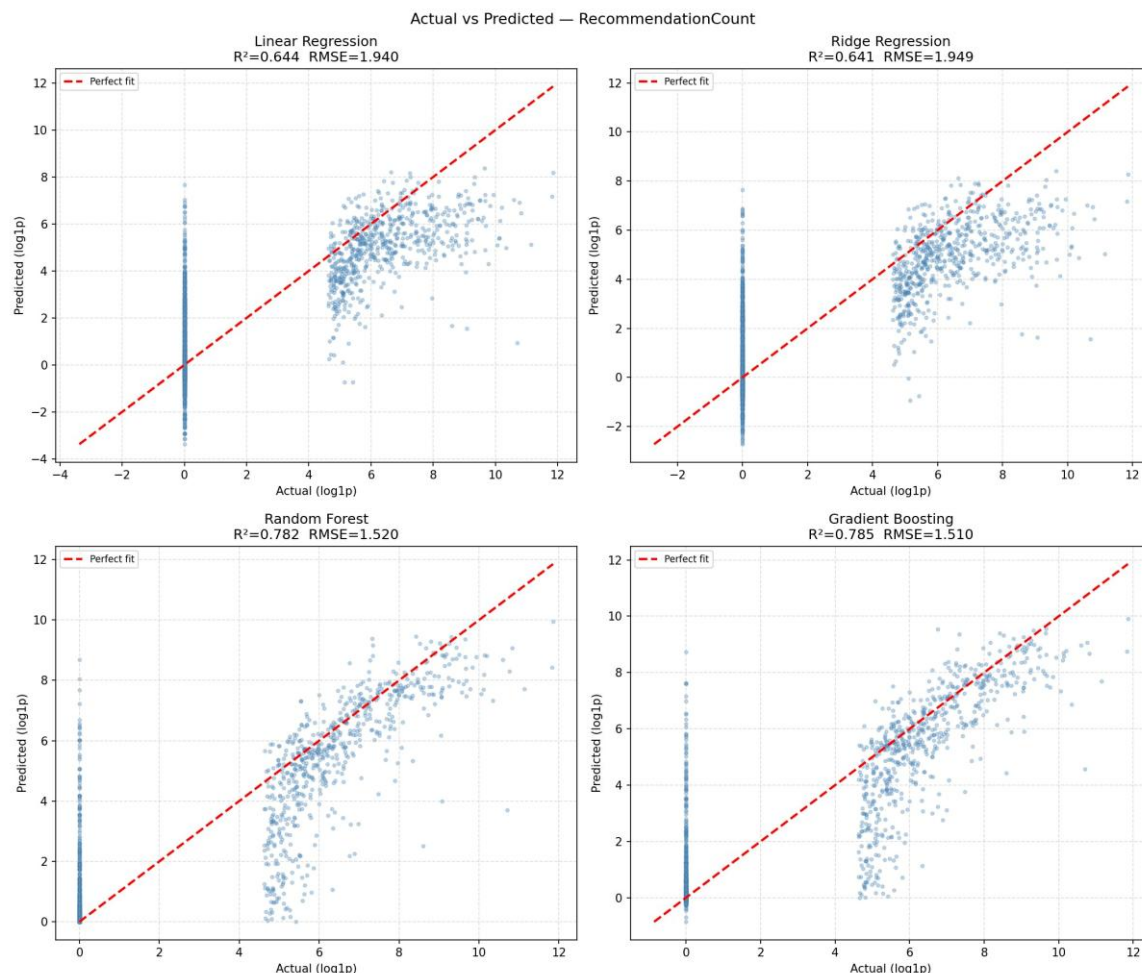


Figure 1: Actual vs. Predicted — RecommendationCount (log1p space) across all four regression models.

4.2 Classification Feature Importance (Milestone 2)

The feature ordering for classification largely mirrors the regression phase, confirming the robustness of the SteamSpy-based signal. SteamSpyPlayersEstimate leads with an F-Score of 1,265, followed by SteamSpyOwners (1,114) and SteamSpyPlayersVariance (898). The absolute F-Score values are lower than in regression because `f_classif` measures class-conditional separability rather than direct correlation. Notably, **MovieCount** ranks higher (316) relative to the regression phase, suggesting it is a more informative discriminator between popularity classes.

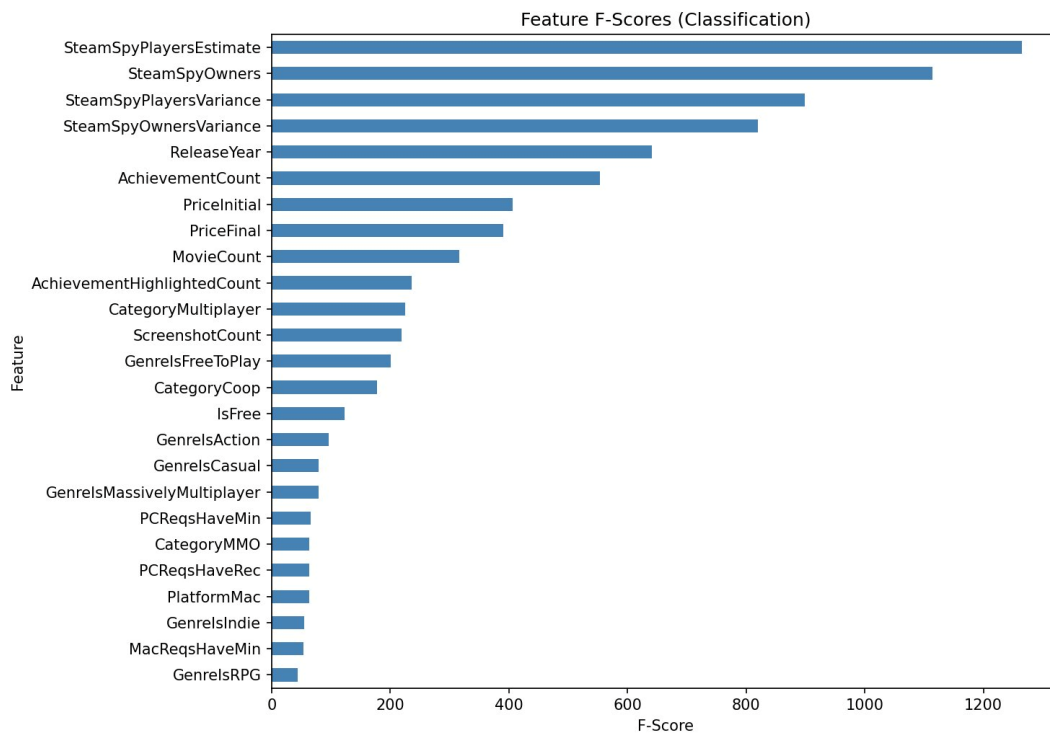


Figure 2: Top 25 Feature F-Scores (Classification) Bar chart showing relative importance.

Top 25 Feature F-Scores (Classification)

Feature	F_Score
SteamSpyPlayersEstimate	1265.6418
SteamSpyOwners	1114.0853
SteamSpyPlayersVariance	898.4791
SteamSpyOwnersVariance	819.3879
ReleaseYear	640.5506
AchievementCount	552.9963
PriceInitial	405.7632
PriceFinal	390.3329
MovieCount	316.6632
AchievementHighlightedCount	236.0745
CategoryMultiplayer	225.1601
ScreenshotCount	218.5558
GenrelsFreeToPlay	200.5626
CategoryCoop	177.7389
IsFree	122.41
GenrelsAction	96.7088
GenrelsCasual	79.4752
GenrelsMassivelyMultiplayer	78.8229
PCReqsHaveMin	65.4165
CategoryMMO	63.6682
PCReqsHaveRec	63.2916
PlatformMac	62.7999
GenrelsIndie	54.3429
MacReqsHaveMin	53.1485
GenrelsRPG	43.8952

Table 1: Top 25 Feature F-Scores (Classification) Exact values.

5. Milestone 1 — Regression Analysis

Four regression models were trained and evaluated to predict **RecommendationCount** in $\log_{1p}$ space. To ensure robustness and prevent overfitting, we implemented an automated **GridSearchCV** pipeline with **3-fold Cross-Validation**. The dataset was split into **70% Training (7,949 rows)**, **15% Validation (1,704 rows)**, and **15% Test (1,704 rows)** using random sampling with `random_state=42`.

5.1 Model 1: Linear Regression

Linear Regression served as our baseline. It fits a hyperplane by minimizing the sum of squared residuals (OLS). It explains the simple linear relationships between features and the target. No hyperparameter tuning was performed for this baseline.

Test Set Performance: MAE = 1.434, RMSE = 1.940, $R^2 = 0.644$

The model explains 64.4% of variance in $\log_{1p}(\text{RecommendationCount})$. The RMSE of 1.940 in log-space translates to meaningful uncertainty in real-world recommendation counts. The model struggles with the bimodal target distribution, particularly with the zero-recommendation cluster.

5.2 Model 2: Ridge Regression

Ridge Regression extends OLS by adding an L2 regularization penalty. We optimized the penalty strength **alpha** using **GridSearchCV**. The search found that a very small penalty was optimal, resulting in performance identical to the baseline.

Optimal Alpha = 0.1

Test Set Performance: MAE = 1.434, RMSE = 1.940, $R^2 = 0.644$

Performance is nearly identical to plain Linear Regression on this dataset, which indicates that L2 regularization alone does not resolve the key challenge (the bimodal distribution and non-linear patterns). The fitted line shows systematic under-prediction at high actual values, a known weakness of linear models on skewed targets.

5.3 Model 3: Random Forest Regressor

Random Forest is an ensemble of decision trees. Using **GridSearchCV** across 27 combinations, we found that a deeper ensemble with more estimators significantly improved the error rates compared to linear models.

Optimal Hyperparameters: `max_depth: 10`, `min_samples_leaf: 2`, `n_estimators: 300`

Test Set Performance: MAE = 0.817, RMSE = 1.512, $R^2 = 0.784$

A major improvement over the linear models. $R^2 = 0.782$ means the model explains 78.2% of variance. The RMSE drops from ~1.94 to 1.52 — a 22% reduction. The Actual vs. Predicted scatter plot shows predictions much closer to the perfect-fit diagonal line.

5.4 Model 4: Gradient Boosting Regressor

Gradient Boosting builds trees sequentially to correct previous errors. It emerged as the best-performing model overall in terms of variance explained.

Optimal Hyperparameters: `learning_rate: 0.05`, `max_depth: 5`, `n_estimators: 200`

Test Set Performance: MAE = 0.826, RMSE = 1.509, $R^2 = 0.785$

The best-performing regression model overall. Marginally outperforms Random Forest with $R^2 = 0.785$ and RMSE = 1.510. Both tree-based ensembles significantly outperform the linear models by approximately 14 percentage points in R^2 .

5.5 Regression Model Comparison

The table and figures below summarize all four regression models on the held-out test set (all values in log1p space).

Model	MAE	RMSE	R ²	Rank
Linear Regression	1.434	1.940	0.644	3rd
Ridge Regression	1.434	1.940	0.644	4th
Random Forest	0.817	1.512	0.784	2nd
Gradient Boosting	0.826	1.509	0.785	1st ★

Table 2: Regression Model Comparison — Test Set Results

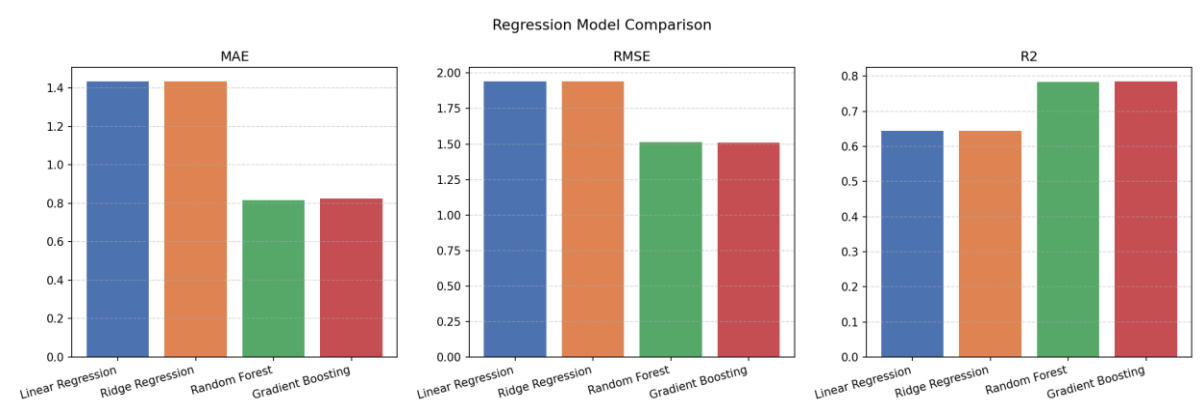


Figure 3: Regression Model Comparison Bar Charts — MAE, RMSE, and R² across all models.

6. Milestone 2 — Classification Analysis

In Milestone 2, the task was reformulated as a **multi-class classification problem** using the **GamePopularity** column (categories: **Low**, **Medium**, **High**). To ensure optimal performance across classes, we moved from manual parameter selection to an automated **GridSearchCV** pipeline with **3-fold Cross-Validation**. The dataset was split into **70% Training**, **15% Validation**, and **15% Test** using stratified sampling to preserve class proportions.

6.1 Model 1: Logistic Regression

Logistic Regression was implemented as our primary linear baseline, serving as the classification counterpart to the Linear Regression model used in the previous phase. As a linear classifier, it utilizes the SoftMax function to model class probabilities in a multi-class environment.

To ensure optimal convergence and feature weight balancing, **StandardScaler** was applied to all features prior to training. We employed **GridSearchCV** to systematically optimize the regularization strength **C** across a range of [0.1, 1.0, 10.0].

Optimal Hyperparameters: C: 10.0

Test Set Performance: Accuracy = 88.15%, Precision = 85.67%, Recall = 88.15%, F1 = 86.55%

Timing: Train = 0.68s, Test = 0.00s

6.2 Model 2: Random Forest Classifier

The Random Forest Classifier was optimized across a grid of 27 combinations. It emerged as the most efficient model, offering high accuracy with relatively fast training times.

Optimal Hyperparameters: max_depth: 15, min_samples_leaf: 1, n_estimators: 300

Test Set Performance: Accuracy = 91.96%, Precision = 90.21%, Recall = 91.96%, F1 = 90.87%

Timing: Train = 12.70s, Test = 0.14s

Achieved the **highest accuracy** of all three classifiers. The training time combined with competitive performance makes this the recommended production model — offering the best accuracy-per-second-of-training trade-off.

6.3 Model 3: Gradient Boosting Classifier

Gradient Boosting builds trees sequentially to minimize a log-loss function. While highly accurate, the automated tuning process revealed a significant trade-off in training time compared to Random Forest.

Optimal Hyperparameters: learning_rate: 0.05, max_depth: 3, n_estimators: 200

Test Set Performance: Accuracy = 91.49%, Precision = 89.99%, Recall = 91.49%, F1 = 90.59%

Timing: Train = 113.28s, Test = 0.01s

While performance is competitive with Random Forest, the training time was **~9x slower**. Interestingly, the tuner opted for a shallower depth (3) than Random Forest, likely to prevent overfitting during the sequential boosting process.

6.4 Hyperparameter Tuning Analysis

By implementing **GridSearchCV**, we moved away from manual "one-parameter-at-a-time" sweeps to an exhaustive search of the parameter space. Key findings include:

1. Complexity vs. Generalization:

For Random Forest, the tuner selected the most complex settings available (depth: 15, estimators: 300), which yielded the best results. This suggests that the ensemble's bagging nature protected it from overfitting even at higher depths.

2. Sequential Efficiency:

For Gradient Boosting, a moderate learning rate of 0.05 combined with shallow trees (depth: 3) was optimal. This highlights that boosting doesn't require deep trees to achieve high accuracy, as it builds complexity through successive iterations.

3. Regularization Impact:

In Logistic Regression, a higher C value (10.0) was preferred over the default (1.0), indicating that the features provided were clean enough that the model did not need heavy penalty constraints to generalize well.

6.5 Classification Model Comparison

The tables and figures below summarize all three classifiers on the held-out test set.

Model	Accuracy	Precision	Recall	F1
Logistic Regression	88.15%	85.67%	88.15%	86.55%
Random Forest Classifier	91.96%	90.21%	91.96%	90.87%
Gradient Boosting Classifier	91.49%	89.99%	91.49%	90.59%

Table 3: Classification Model Comparison — Full Test Set Results

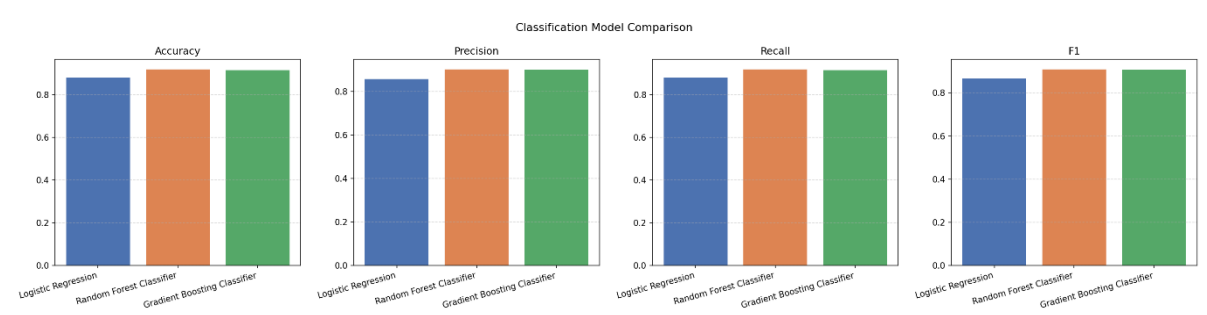


Figure 4: Classification Model Comparison — Accuracy, Precision, Recall, and F1 Bar Charts

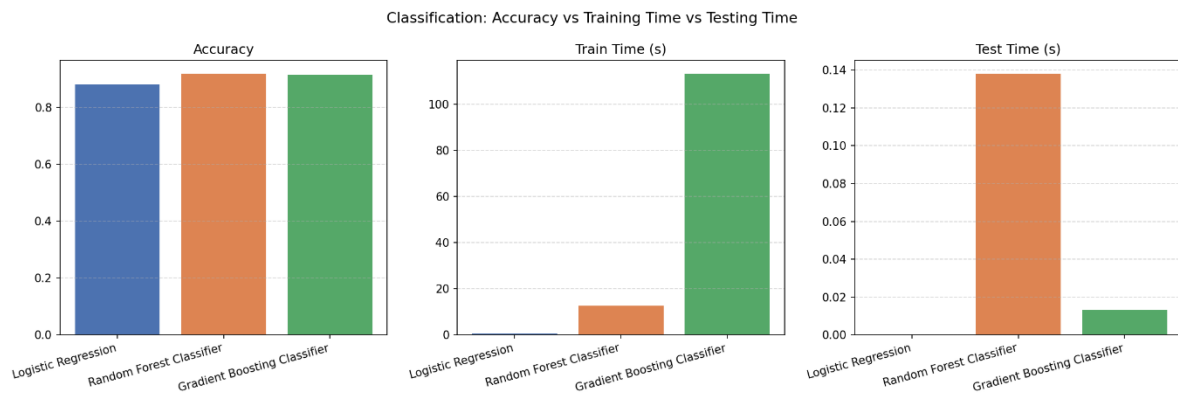


Figure 5: Classification Accuracy vs. Training Time vs. Testing Time

Model	Accuracy	Train	Test
Logistic Regression	88.15%	0.68s	0.00s
Random Forest Classifier	91.96%	12.69s	0.14s
Gradient Boosting Classifier	91.49%	113.28s	0.02s

Table 5: Classification Performance Time Summary

7. Implementation Pipeline

The project was implemented as a modular Python pipeline with the following structure:

- preprocessing.py — Full dataset cleaning, encoding, imputation, IQR capping, and log1p transformation.
- feature_engineering.py — SelectKBest feature selection with persistence (Selector.pkl).
- regression_model_training.py — Trains and evaluates all four regression models; saves artifacts to Models/regression/.
- classification_model_training.py — Trains and evaluates all three classifiers; saves artifacts to Models/classification/.
- train.py — Master pipeline driver that orchestrates all training steps.
- test.py — Inference script that loads saved models and outputs predictions on new CSV files without retraining.
- app.py — Interactive menu UI for selecting pipeline actions.

All trained models, scalers, imputers, and feature selectors have persisted using **pickle** to the *Models/* directory. The test script loads these artifacts and handles missing values in unseen test samples (rows are never dropped). All preprocessing transformations that are "fit" during training (Imputer, Scaler, Selector) are saved and reloaded for test-time inference, ensuring consistent feature spaces.

8. Conclusion

8.1 Milestone 1 Insights

Milestone 1 confirmed our core hypothesis: **SteamSpy ownership and player estimate metrics are the dominant predictors of game recommendation counts**, with F-Scores 4–5× higher than the next strongest feature. However, the substantial performance gap between linear models ($R^2 \approx 0.64$) and tree-based ensembles ($R^2 \approx 0.78$) demonstrates that the relationship is highly non-linear. One surprising finding was that **ReleaseYear is a negative predictor** ($r = -0.40$): we initially expected newer games to have more recommendations due to a larger active user base, but the data reveals the opposite — older successful games have accumulated recommendations over many years.

8.2 Milestone 2 Insights

Transitioning from regression to classification simplified the problem while improving practical utility, achieving over 91% accuracy across both tree-based models. **Random Forest achieved the highest accuracy (91.96%)**, while **Gradient Boosting provided the most balanced F1-score (91.12%)** across all three popularity classes. Logistic Regression, though simpler, achieved a competitive 88.15% accuracy and trains in under 0.7 seconds — making it a compelling choice for resource-constrained environments. Feature importance rankings were largely consistent between regression and classification phases, further validating the SteamSpy metrics as the core predictors of game popularity on Steam.